

Programação em C#
Prof. Pablo R. Gonçalves
Acesso a banco de dados usando padrão Repository

```
// Classe de modelo
public class Cliente
{
    public int IdClientes { get; set; }
    public string Nome { get; set; }
    public string Endereco { get; set; }
    public string Telefone { get; set; }
}

// Classe de repositório
public class ClienteRepository
{
    private readonly string _connectionString;

    public ClienteRepository()
    {
        _connectionString =
        ConfigurationManager.ConnectionStrings["MySQLConnectionS
tring"].ToString();
    }

    public bool Inserir(Cliente cliente)
    {
        using (MySqlConnection conexao = new
        MySqlConnection(_connectionString))
        {
            try
            {
                conexao.Open();

                string sql = "INSERT INTO Clientes (IdClientes, Nome,
                Endereco, Telefone) " +
                "VALUES (@vId, @vNome, @vEndereco,
                @vTelefone)";

                MySqlCommand comando = new MySqlCommand(sql,
                conexao);
                comando.Parameters.AddWithValue("vId",
                cliente.IdClientes);
                comando.Parameters.AddWithValue("vNome",
                cliente.Nome);
```

```
                comando.Parameters.AddWithValue("vEndereco",
                cliente.Endereco);
                comando.Parameters.AddWithValue("vTelefone",
                cliente.Telefone);

                return comando.ExecuteNonQuery() > 0;
            }
            catch
            {
                throw;
            }
        } // O using cuida do fechamento da conexão
        automaticamente
    }
}

// Código do botão
private void btnSalvar_Click(object sender, EventArgs e)
{
    try
    {
        var cliente = new Cliente
        {
            IdClientes = int.Parse(textID.Text.Trim()),
            Nome = textNome.Text.Trim(),
            Endereco = textEndereco.Text.Trim(),
            Telefone = textFone.Text.Trim()
        };

        var repository = new ClienteRepository();
        bool sucesso = repository.Inserir(cliente);

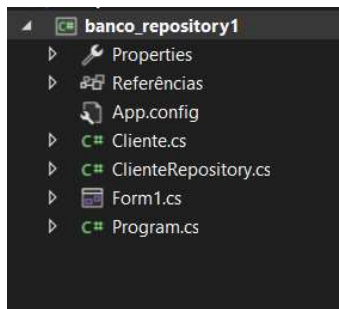
        if (!sucesso)
            MessageBox.Show("Erro ao inserir");
        else
            MessageBox.Show("Inserido com sucesso");
    }
    catch (Exception ex)
    {
        MessageBox.Show("Erro: " + ex.Message, "Erro");
    }
}
```

Usando arquivos separados

Quando separamos nosso código em diferentes arquivos, estamos aplicando o princípio de separação de responsabilidades. Isso traz várias vantagens:

1. **Organização mais clara:** Cada arquivo se concentra em uma única responsabilidade, tornando mais fácil encontrar código específico.
2. **Manutenção simplificada:** Quando precisar modificar a estrutura do modelo Cliente, você saberá exatamente onde ir, sem precisar navegar por um arquivo grande com múltiplas responsabilidades.
3. **Colaboração em equipe:** Diferentes membros da equipe podem trabalhar em diferentes partes do sistema simultaneamente com menos conflitos.
4. **Reutilização:** As classes em arquivos separados são mais facilmente reutilizáveis em outras partes do sistema ou em projetos futuros.
5. **Testabilidade:** Código bem separado é geralmente mais fácil de testar unitariamente.

Crie a seguinte estrutura



Arquivo Cliente.cs

```
using System;
namespace WindowsFormApp1
{
    public class Cliente
    {
        public int IdClientes { get; set; }
        public string Nome { get; set; }
        public string Endereco { get; set; }
        public string Telefone { get; set; }
    }
}
```

Arquivo ClienteRepository

```
using System;
using System.Configuration;
using MySql.Data.MySqlClient;

namespace WindowsFormApp1
{
    public class ClienteRepository
    {
        private readonly string _connectionString;

        public ClienteRepository()
        {
            _connectionString =
                ConfigurationManager.ConnectionStrings["MySQLConnectionS
                tring"].ToString();
        }

        public bool Inserir(Cliente cliente)
        {
            using (MySqlConnection conexao = new
                MySqlConnection(_connectionString))
            {
                try
                {
                    conexao.Open();
                }
            }
        }
    }
}
```

```
        string sql = "INSERT INTO Clientes (IdClientes, Nome,
        Endereco, Telefone) " +
            "VALUES (@vId, @vNome, @vEndereco,
            @vTelefone)";

        MySqlCommand comando = new
            MySqlCommand(sql, conexao);
        comando.Parameters.AddWithValue("vId",
            cliente.IdClientes);
        comando.Parameters.AddWithValue("vNome",
            cliente.Nome);
        comando.Parameters.AddWithValue("vEndereco",
            cliente.Endereco);
        comando.Parameters.AddWithValue("vTelefone",
            cliente.Telefone);

        return comando.ExecuteNonQuery() > 0;
    }
    catch
    {
        throw;
    }
}
```

Arquivo Form1.cs

```
using System;
using System.Windows.Forms;

namespace WindowsFormApp1
{
    public partial class FormClientes : Form
    {
    }
}
```

```
private readonly ClienteRepository _clienteRepository;

public FormClientes()
{
    InitializeComponent();
    _clienteRepository = new ClienteRepository();
}
```

```

private void btnSalvar_Click(object sender, EventArgs e)
{
    try
    {
        var cliente = new Cliente
        {
            IdClientes = int.Parse(textID.Text.Trim()),
            Nome = textNome.Text.Trim(),
            Endereco = textEndereco.Text.Trim(),
            Telefone = textFone.Text.Trim()
        };

        bool sucesso = _clienteRepository.Inserir(cliente);
    }
}

```

```

        if (!sucesso)
            MessageBox.Show("Erro ao inserir");
        else
            MessageBox.Show("Inserido com sucesso");
    }
    catch (Exception ex)
    {
        MessageBox.Show("Erro: " + ex.Message, "Erro");
    }
}
}

```

No arquivo App.config, inserir a string de conexão:

```

<connectionStrings>
    <add name="MySQLConnectionString" connectionString="Server=localhost;
        Database=teste;Uid=root;Pwd=root;Connect Timeout=30;" />
</connectionStrings>

```